Machine Learning in Engineering Science

Week 12: LLMs, bounded agents, and vibe coding

Chi-Hua Jonny Yu / NCKU + LAiMM

Fall 2026 / 2026-11-23

Chapter 1

LLMs, bounded agents, and vibe coding

Week 12: 2026-11-23, Monday 14:00–17:00.

Train a transparent character bigram language model, then inspect a separate deterministic proposal validator. Neither the bigram nor the rule dispatcher is a Transformer or a live LLM agent.

Learning outcome

Explain next-token prediction; validate structured tool proposals; retain control outside the model.

The meeting contains three 60-minute modules: 60 minutes explanation and 120 minutes practice. The eight core exercises and two optional challenges are in-class practice, not additional weekly graded homework.

1.1 H1: From next-token prediction to tool proposals

1.1.1 Language is an interface, not an oracle

Generate a proposal, then verify the claim outside the model.

Worked reasoning. Token probabilities; sampling; structured proposals; validation; Task 1 prediction.

Check. What could make fluent output incorrect?

Explanation and boundary. A likely token sequence can lack factual evidence, correct arithmetic, or permission. Separate these checks.

1.1.2 A token is not necessarily a word

A tokenizer maps text to integer IDs using a specified vocabulary.

Worked reasoning. Character demo: hot = h,o,t; a production subword tokenizer can split differently. Preserve tokenizer identity.

Check. Can one word always be counted as one token?

Explanation and boundary. No. Token boundaries depend on the tokenizer and text. The notebook uses characters, not a production LLM tokenizer.

1.1.3 Autoregression factorizes a sequence

Each conditional distribution uses the preceding context only.

Worked reasoning. p(A)=0.5; p(B given A)=0.6; p(C given AB)=0.2; joint probability=0.06.

Check. Does the factorization prove the sequence is true?

Explanation and boundary. No. It describes sequence probability under a model, not truth. The multiplication follows the chain rule.

1.1.4 Cross-entropy penalizes surprise

Training lowers negative log probability on observed next tokens.

Worked reasoning. If p=0.8 then -ln(p)=0.223; if p=0.2 then loss=1.609. Mean=0.916 nats.

Check. Why is assigning low probability to the observed token costly?

Explanation and boundary. Negative log grows as probability approaches zero. Mean teacher-forced loss is not factual verification.

1.1.5 Attention selects context-dependent mixtures

Causal masking prevents a position from reading later tokens.

Worked reasoning. At position 2, keys 1 and 2 are allowed; keys 3 and 4 are blocked before softmax.

Check. Does attention provide an external source of facts?

Explanation and boundary. No. It mixes representations from supplied context. Retrieved evidence and its quality remain separate responsibilities.

1.1.6 Pretraining and post-training differ

Pretraining learns broad prediction; post-training changes response behavior.

Worked reasoning. A polite response with well-formed JSON can contain an incorrect number or an unsupported citation.

Check. Does instruction tuning remove the need for tests?

Explanation and boundary. No. Behavior adaptation is not a guarantee of accuracy, authorization, or grounding.

1.1.7 Sampling changes a distribution

Temperature divides logits before softmax; it is not a truth control.

Worked reasoning. Logits [2,0]: T=1 gives [0.881,0.119]; T=2 gives [0.731,0.269].

Check. What does increasing temperature change here?

Explanation and boundary. It flattens this distribution and raises the lower-logit token probability. It does not make the content more or less factual by definition.

1.1.8 A tiny language model is inspectable

Character bigrams estimate the next character from one previous character.

Worked reasoning. Run the fixed original teaching corpus. Find the most frequent successor of a space and compare its count with its probability.

Check. Is this a miniature Transformer?

Explanation and boundary. No. It is a trained conditional-count language model with one-character context; it lacks attention and contextual embeddings.

1.1.9 Prediction before sampling creates evidence

Keep corpus, vocabulary, smoothing and random seed with the generated text.

Worked reasoning. Predict whether lower temperature concentrates probabilities; compare entropy and sampled continuations without changing the corpus.

Check. Must a lower-entropy model produce a better answer?

Explanation and boundary. No. Entropy measures distribution concentration. Quality needs a task-specific evaluation independent of sampling sharpness.

1.1.10 Summary: separate probability and proof

A language model proposes tokens; verified tools establish bounded numeric results.

Worked reasoning. Show one next-character distribution, one computed loss and one limitation of the bigram model.

Check. Which component will authorize the next tool call?

Explanation and boundary. Deterministic validation outside the model. A model probability cannot grant permission.

1.2 H2: Structured outputs and bounded tool execution

1.2.1 The tool boundary is executable code

A proposal becomes an action only after deterministic validation.

Worked reasoning. Tasks 3-5: valid JSON, invalid JSON, tool allowlist, numeric checks and a finite attempt budget.

Check. Why not execute a plausible tool name directly?

Explanation and boundary. A tool name is untrusted data. Unknown names must be rejected without dynamic evaluation or dispatch to arbitrary code.

1.2.2 JSON syntax is only the first gate

Successful parsing establishes a data structure, not its meaning.

Worked reasoning. Allowed: tool=mean with finite values. Not allowed: tool=delete_files even when the JSON parses.

Check. Can a structured-output guarantee replace permissions?

Explanation and boundary. No. Syntax, schema conformance, semantics and authorization are distinct guarantees.

1.2.3 Use exact fields and explicit types

Reject booleans, NaN, empty arrays and oversized arrays before calculation.

Worked reasoning. In Python isinstance(True,int) is true; the validator uses type(x) in (int,float) and a finite check.

Check. Why reject a Boolean in numeric readings?

Explanation and boundary. Its programming-language subtype does not establish the intended measurement meaning. The declared contract accepts numeric readings only.

1.2.4 No natural-language sentence grants tools

A retrieved instruction remains content, even when it looks urgent.

Worked reasoning. Try delete_files and a values array containing 'ignore rules'. Verify no external side effect occurs.

Check. What is the enforcement point?

Explanation and boundary. The allowlist and schema validator, not a promise in the system prompt. These probes cover only the supplied implementation and cases.

1.2.5 Numerical results need their own checks

Valid inputs can still overflow a calculation; validate the result too.

Worked reasoning. Test the bounded mean implementation with large finite inputs; it must return a finite result or an explicit rejection.

Check. Does a valid schema prove the arithmetic is correct?

Explanation and boundary. No. Independently check arithmetic against small fixtures and reject nonfinite outputs rather than silently serializing them.

1.2.6 A finite budget belongs to the harness

Count attempts before dispatch; rejected proposals still consume an attempt.

Worked reasoning. Submit three proposals. The third must be budget_exhausted and must not invoke a tool.

Check. Why count rejected attempts?

Explanation and boundary. Otherwise repeated invalid proposals can loop forever without spending the nominal tool budget.

1.2.7 Trace observable events, not hidden thoughts

Store proposal, validation status, output and stop reason.

Worked reasoning. Locate the proposal that failed and the code that blocked it. A trace is an observable execution record, not private chain-of-thought.

Check. Can a detailed trace establish truth by itself?

Explanation and boundary. No. A trace improves inspection; incorrect code can log incorrect results consistently.

1.2.8 An LLM adapter is optional, not required

Manual or captured JSON proposals exercise the same boundary at zero inference cost.

Worked reasoning. Use a well-formed and a malformed response. Record parse success, validation success and numeric correctness separately.

Check. Are we evaluating a real LLM in the core notebook?

Explanation and boundary. No. It trains a small count model and uses explicit tool proposals. Optional external outputs must be labeled with model and date and evaluated separately.

1.2.9 Vibe coding needs acceptance criteria

Ask for a small change with inputs, forbidden behavior and tests.

Worked reasoning. Add median as an allowed tool; preserve mean behavior; reject unknown tools; add odd/even/empty cases.

Check. Why is 'make this agent smarter' insufficient?

Explanation and boundary. It does not specify observable behavior or a stopping condition. Students must review actual code and preserve the reference behavior.

1.2.10 Summary: the model never owns permissions

Parsing, validation, execution and recording are separate stages.

Worked reasoning. Save a valid result, a rejection and a budget stop. Keep the original proposal alongside each event.

Check. What evidence must survive the runtime?

Explanation and boundary. Inputs, contract, source, versions, trace and exported results. A successful chat screenshot is insufficient.

1.3 H3: Tool-contract studio: test, explain, export

1.3.1 Build a proposal-to-evidence artifact

Finish eight core tasks before attempting optional changes.

Worked reasoning. Restore 5 min; core tasks 25 min; peer checks 20 min; export and exit 10 min.

Check. What do you restore after a runtime reset?

Explanation and boundary. Run the notebook top to bottom; do not paste a remembered result into an evidence record.

1.3.2 Task 1: read a learned distribution

Compare successor counts with add-one probabilities for one context character.

Worked reasoning. With vocabulary size V and context count N, add-one assigns 1/(N+V) to an unseen successor.

Check. What did smoothing assume?

Explanation and boundary. A symmetric pseudo-count over the vocabulary. It avoids zero probability but is not evidence that the successor was observed.

1.3.3 Task 2: compare sampling controls

Record entropy for the same logits at two positive temperatures.

Worked reasoning. Use a fixed seed per run and preserve the probability vectors, not just the sampled strings.

Check. Can two different temperatures produce the same short string?

Explanation and boundary. Yes. One sampled continuation is a noisy observation and does not identify the underlying distribution.

1.3.4 Task 3: reject the unknown capability

Send an unknown tool name to the same dispatcher.

Worked reasoning. Record expected status before calling the dispatcher. Verify that no accepted result is produced.

Check. Should the validator silently rename an unknown tool?

Explanation and boundary. No. Silent repair changes the requested operation. Return a specific failure and require an explicit new proposal.

1.3.5 Task 4: classify malformed readings

Test empty, Boolean, string, NaN and infinite values separately.

Worked reasoning. The JSON parser rejects nonstandard constants; numeric validation also rejects nonfinite native Python values.

Check. Why keep separate tests for text and Python objects?

Explanation and boundary. They enter through different boundaries. Passing one path does not prove the other path uses the same checks.

1.3.6 Task 5: stop a repeated proposal loop

A cap counts every attempt, not only successful actions.

Worked reasoning. A cap of two permits at most two validations/dispatch attempts; the third event records a budget stop.

Check. What would happen if only successes counted?

Explanation and boundary. A sequence of malformed proposals could make no progress indefinitely while never reaching the cap.

1.3.7 Task 6: repair syntax without changing intent

Correct malformed JSON manually and rerun the exact same contract.

Worked reasoning. Repair a trailing comma. Explain why the fixed string still needs semantic validation.

Check. Does syntax repair justify executing a newly added tool?

Explanation and boundary. No. A repair may change meaning. Compare fields and retain the allowlist regardless of parse success.

1.3.8 Task 7: expose arithmetic with a fixture

Check mean([1,2,6]) = 3 before reading a model explanation.

Worked reasoning. A zero-mean fixture can hide a wrong denominator; [1,2,6] makes the expected result informative.

Check. What does one passing fixture establish?

Explanation and boundary. Agreement on that fixture only. Add empty, large, negative and length-bound cases before broader claims.

1.3.9 Task 8: hand over the evidence ZIP

Export eight predictions, eight explanations and three reflections.

Worked reasoning. Blank answers yield draft status. Filled fields mean ready for discussion, not automatically correct.

Check. What should a partner inspect first?

Explanation and boundary. Read the contract and code before running it; compare actual results with independently calculated fixtures.

1.3.10 Summary: extend only a declared boundary

Optional A adds median; optional B tests proposal formatting variants.

Worked reasoning. Explain one observation for each concept and one limitation. Keep the original reference unchanged.

Check. Why are these three different questions?

Explanation and boundary. Token likelihood describes generation, permission controls actions, and correctness requires task evidence. None implies the others.

1.4 Mathematical reference

$$p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{<t}), \qquad L = -\frac{1}{T}\sum_{t=1}^{T} \log p(x_t \mid x_{<t}).$$

$$P_{ij} = \frac{C_{ij} + 1}{\sum_k C_{ik} + V}, \qquad p_i(T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}, \quad T > 0.$$

The count model uses one-character context, not Transformer attention. Likelihood and entropy are not measures of factual support.

1.5 Eight core practice tasks

1.5.1 Task 1: Inspect a learned next-character distribution

Predict the most frequent successor of a space. Read the actual transition counts, then compare the normalized add-one probabilities.

Self-check. All successor probabilities, not just the six plotted, sum to one. A smoothed nonzero probability does not imply an observed transition.

1.5.2 Task 2: Change temperature without changing the model

Predict how T=0.5 and T=2 change entropy. The corpus and trained counts stay fixed; compare distributions as well as generated samples.

Self-check. T must be finite and positive. Entropy concentration is not a factual-accuracy measure.

1.5.3 Task 3: Reject an unknown tool before execution

Predict statuses for a mean proposal and a forbidden capability. No files, network requests or real messages are touched.

Self-check. The forbidden proposal produces no numeric result. No eval, shell tool or dynamic import is used to dispatch a name.

1.5.4 Task 4: Probe malformed numeric inputs

Predict a reason for each rejection before running. Python values and text JSON use different entrance gates.

Self-check. All six malformed native inputs reject. Nonstandard JSON NaN is also rejected by the parser in Task 6.

1.5.5 Task 5: Exhaust the attempt budget

Predict the third status when two proposals have already consumed the two-attempt cap. Rejected proposals consume attempts too.

Self-check. Only two proposals enter dispatch. The third is a stop event, not a third tool attempt.

1.5.6 Task 6: Repair syntax while preserving the boundary

Compare a valid proposal, a trailing comma, a duplicate key and NaN text. Repair only the syntax of a copied string.

Self-check. Duplicate fields are not silently overwritten. Syntax repair still passes through the same validator.

1.5.7 Task 7: Independently check arithmetic

Calculate 1+2+6 by hand, divide by 3, then check the tool. Explain why zero-mean inputs might hide a denominator bug.

Self-check. The expected 3.0 was written independently, not generated by dispatch. One fixture is not a full arithmetic evaluation.

1.5.8 Task 8: Audit what the exported record can support

Check every task is present, preserve a corpus hash and inspect one rejection with its original proposal.

Self-check. The final export records incomplete reflections as draft. No model account or paid quota is needed.

1.6 Evidence handover and summary

Explain next-token prediction; validate structured tool proposals; retain control outside the model. Save eight predictions, eight explanations and three reflections. Preserve source, data, versions, traces and one limitation. The notebook exports a draft when fields are blank; response completeness does not establish correctness. Save the edited notebook separately if changing code.

1.7 Primary references

- Attention Is All You Need: https://arxiv.org/abs/1706.03762.
- Building effective agents: https://www.anthropic.com/engineering/building-effective-agents.